

Introduction

Theory of Computation/Theoretical Computer Science

- The theory of computation is the branch of computer science that deals with how ***efficiently*** problems can be solved on a ***model of computation***, using an ***algorithm***.
 - A ***model of computation*** is the definition of set of allowable operations used in computation and their respective costs.
 - describes how a set of outputs are ***computed*** given a set of inputs.
 - describes how units of computations, memories, and communications are ***organized***.
 - It is used for measuring the ***computational complexity*** of the problem for which it is designed.

Theory of Computation

- Abstract away as many specifics as we can
- All machines are different but basics of computation model remains the same.

Theory of Computation

This field is divided into three major branches:

- ***Automata Theory and Language***

- How can we mathematically model computation?

- ***Computability Theory***

- What problems can be solved by a computer?

- ***Complexity Theory***

- What makes some problems computationally hard and others easy?

Automata Theory

- ***Automata Theory*** is a branch of computer science that deals with ***designing abstract self-propelled computing devices*** that follow a predetermined sequence of operations automatically.
- The word “***Automata***” is plural of “***automaton***” which simply means ***any machine***.
- A machine that ***operates on its own (automatically)*** without the need for human control.
- These machines need ***not to be a physical hardware***.

Contd.

- Through automata, computer scientists are able to understand
 - *how machines compute functions and solve problems.*
 - *What the computer machine could do and what it could not do.*
- In automata we will simulate parts of computers.

Contd.

- Helps in design and construction of different software's and what we can expect from our software's.
- Automata play a major role in theory of computation, compiler design, artificial intelligence.

Families of Automaton

Deals with the definitions and properties of mathematical model of computation.

Example:

- Finite Automata (Simplest)
 - Text Processing
 - Compilers
- Context Free grammars
 - Programming languages
 - AI

Questions

- How does the Computer ***interpret input*** given by input devices? And How does it ***find commands and then carry out appropriate actions.***?
- How does computers ***read and interpret files***, for example when it comes to ***compiling a program*** from a language such as Java, C or ML?
- How does Compilers recognize keywords like if or else?
- How does the search engine searches the keyword or phrase entered by the user?
- How does the search engine recognizes different forms, of string such as type, typing and typed?

Contd.

What if

- Your program had to work across lines (and an arbitrary number of spaces)?
- You had to take into account that the first doubled word might be capitalized?
- The two doubled words might be separated by html tags?

Contd.

- Imagine you had to write code for a vending machine that takes coins and in return delivers variously priced goods. How does the machine keep track of how much money the user has put in so far, and how to produce correct change from the coins it holds? It's your job to keep track of all the cases that may arise.

Contd.

- Imagine you had to write a piece of code that takes as input a file containing a Java program and checks whether the curly brackets {} are all properly balanced.

Characteristics of machines

A typical abstract machine consists of:

- ***Inputs***

- Sequences of symbols selected from a finite set Σ of input signals.
- Set Σ is the set $\{x_1, x_2, x_3 \dots x_k\}$ where k is the number of inputs.

- ***Outputs***

- Sequence of symbols selected from finite set Z .
- set Z is the set $\{y_1, y_2, y_3 \dots y_m\}$ where m is the number of outputs

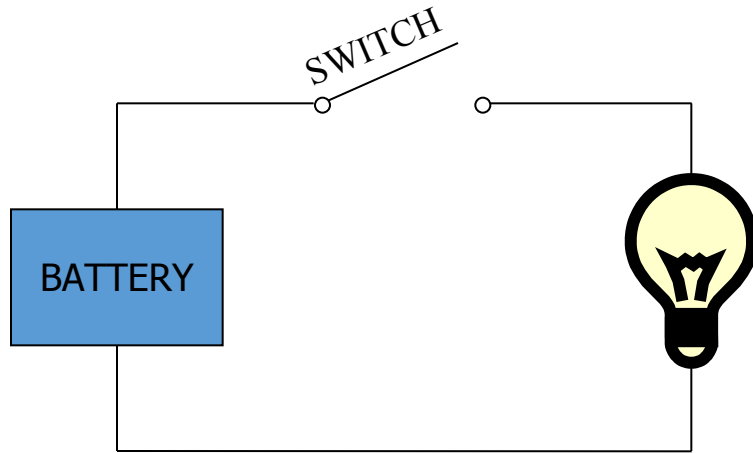
- ***States***

- Finite set Q , whose definition depends on type of automaton.

Cont.

- **Automatons** are abstract models of machines that perform *computations on an input* by moving through *a series of states* or configurations.
- At each state of the computation, a *transition function* determines the next configuration on the basis of a finite portion of the present configuration.
- As a result, once the computation reaches an accepting configuration, it accepts that input.

A simple computer



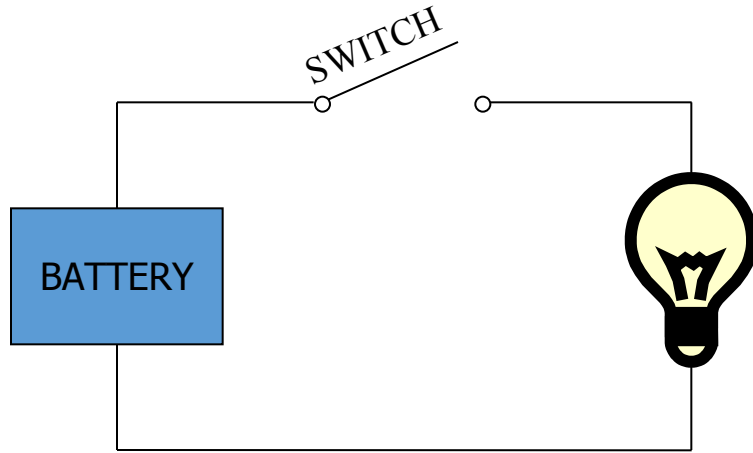
input: switch

output: light bulb

actions: flip switch

states: on, off

A simple “computer”

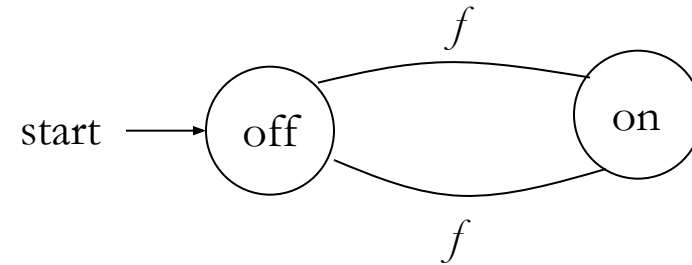


input: switch

output: light bulb

actions: f for “flip switch”

states: on, off



bulb is on if and only if
there was an **odd** number
of flips

Important Concepts in Automata

Introduction to Language

- Automata deals with language.
- Language is combination of
 - Alphabets (Σ)
 - Grammar/Rules

Types of Languages

There are two types of languages

- ***Formal Language***

- A language which can be represented mathematically.

- ***Informal Language***

- A language which cannot be represented mathematically.

Symbol/Letter

- A symbol is a building block for a string.
- Symbols cannot be sub-divided, they are the atoms of everything we build.
- In the theory of formal languages they are usually called letters.
Examples: a, A, 0, 1, %, @.
- We use letters from the end of the Roman alphabet, such as x, y, z to refer to an arbitrary symbol.

Alphabet (Σ)

- A finite ***non-empty set of symbols*** (called letters), is called an alphabet.
- We use capital Greek letters, typically Σ (Greek letter sigma), to refer to an arbitrary alphabet.
- Example:
 - $\Sigma = \{A, B, C, \dots, Z, a, b, c, \dots, z\}$
 - $\Sigma = \{0, 1\}$
 - $\Sigma = \{a, b, c\}$
 - $\Sigma = \{ab, cd\}$
 - $\Sigma = \{\text{for}, \text{int}, \text{while}\}$

Strings

- ***Concatenation of zero or more finite number of letters/symbols*** from the alphabet is called a string.
- Note that every letter can be viewed as a one-letter string.
- We use letters s, t, u to refer to an arbitrary string.
- Example:
 - $\Sigma = \{a, b, c\}$ then strings are
 - $a, ab, aabbcc, abc, aaaabbbababa$ etc

Empty String/ Null String

- Empty String is ***a string with no symbol i.e. 0 symbol*** which is

denoted by

- Small Greek letter Lambda (λ) or
- Capital Greek letter Lambda (Λ)
- Small Greek Letter Epsilon (ϵ)

Words

- Words are strings belonging to some language i.e. having some *meaning in that language*
- Example
 - $\Sigma = \{A, B, C, \dots, Z, a, b, c, \dots, z\}$
 - Strings : abc, def, ghi, ...
 - Words : The, walk, intelligent , ...

Note:

- All words are strings, but not all strings are words.

Concatenation

- ***Concatenation*** is the operation we perform on words (or letters) ***to obtain longer words.***
- When concatenating ***a*** with ***b*** we get ***ab***, and we can concatenate that with the word ***ba*** to obtain ***abba***.
- If we concatenate 0 letters we get the word ϵ .
- When we concatenate any word with the word ϵ we obtain the same word.

Powers

- We use the notation of powers for concatenation as follows:
- If s is a word then $(s)^n$ is the word we get by concatenating n copies of s .
- For example,

$$(010)^3 = 010010010$$

$$(1)^2 = 11,$$

$$(b)^0 = \epsilon \text{ and}$$

$$(c)^1 = c$$

Powers of an alphabet

Let Σ be an alphabet.

- Σ^k = the set of all strings of length k
- ***Set of all strings***
 - $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$
- ***Set of all non empty strings***
 - $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

Power of an alphabet

- $\Sigma = \{0,1\}$
- $\Sigma^0 = \{\Lambda\}$ //strings of length 0
- $\Sigma^1 = \{0,1\}$ // strings of length 1
- $\Sigma^2 = \{00,01,10,11\}$ //strings of length 2
- $\Sigma^3 = \{000,001,010,011, 100,101,110,111\}$ //strings of length 3

String Tokenization

- Tokenization is the act of ***breaking up a sequence of strings*** into piece such as ***words, keywords, phrases, symbols*** and other elements called ***token***.

Example:

- If $\Sigma = \{a, b, c\}$
 - $s = ababc$
 - Tokenization = $(a)(b)(a)(b)(c)$
- $\Sigma = \{ab, cd\}$
 - $s = abababcd$
 - Tokenization = $(ab)(ab)(ab)(cd)$

Valid/Invalid Alphabet

- While defining an alphabet of letters consisting of more than one symbols, ***no letter should be started with the letter of the same alphabet i.e.*** one letter should not be the prefix of another. However, a letter may be ended in a letter of same alphabet.
- $\Sigma_1 = \{B, aB, bab, d\}$
- $\Sigma_2 = \{B, Ba, bab, d\}$
- Σ_1 is a valid alphabet while Σ_2 is an in-valid alphabet.

Example

- $\Sigma_2 = \{B, Ba, bab, d\}$ and $s = BBad$
- This string can be tokenized in two different ways.
 - $(B)(Ba)(d)$
 - $(B)(B)(ad)$ cannot be identified as string of Σ_2

Valid/In-valid alphabets

- As when this string is scanned by the ***compiler (Lexical Analyzer)***, first symbol B is identified as a letter belonging to Σ , while for the second letter the lexical analyzer would not be able to identify, so while defining an alphabet it should be kept in mind that ambiguity should not be created.

Note:

Alphabet that cause ambiguity is not a valid alphabet.

Length of String

- The length of string s , denoted by $|s|$, is the number of letters in the string.
- Always positive.

Example

- If $\Sigma = \{a, b\}$
 - $s = abba$
 - Tokenization = $(a)(b)(b)(a)$
 - $|s| = 4$

Example Contd.

- $\Sigma = \{ab, bab\}$
 - $s = abbabbababbab$
 - Tokenization = $(ab)(bab)(bab)(ab)(bab)$
 - $|s| = 5$
- $\Sigma = \{for, int\}$
 - $s = intforint$
 - Tokenization = $(int)(for)(int)$
 - $|s| = 3$

Reverse of String

- The reverse of a string s denoted by **rev(s)** or s' , is obtained by writing the letters of s *in reverse order*.
- First do tokenization then reverse the string.

Example

if $s = \text{abababc}$ is a string defined over $\Sigma = \{\text{ab}, \text{c}\}$ then

Tokenization = $(\text{ab})(\text{ab})(\text{ab})(\text{c})$

$|s| = 4$

$\text{rev}(s) = \text{cababab}$

Language

- A language is a collection of words, which we think of as a set.
- Examples
 - $\{\epsilon\}$
 - $\emptyset$
 - $\{ab, abc, aba\}$
 - $\{(a)^n \mid n \in \mathbb{N}\}$.
- We use letters such as L , L_1 and L^1 to refer to an arbitrary language.

Types of Languages

- **Single Letter Language**

- Language whose alphabet is having only one letter.
 - $\Sigma = \{a\}$
 - $\Sigma = \{b\}$
 - $\Sigma = \{\wedge\}$

- **Finite Language**

- Which generates countable strings

- **Infinite Language**

- Which generates infinite strings

Defining new languages from old ones

- **Union:** Since languages are just sets we can form their unions.
- **Intersection:** Since languages are merely sets we can form their intersections.
- **Set difference:** If L_1 and L_2 are languages we can form
$$L_1 \setminus L_2 = \{s \in L_1 \mid s \notin L_2\}$$
- **Complement :** If L is the language of all words over some alphabet Σ , and L_0 is a subset of L then the complement of L_0 in L is $L \setminus L_0$, the set of all words over Σ which are not contained in L_0 .

Defining new languages from old ones

- **Concatenation:** Because we can concatenate words we can use the concatenation operation to define new languages from existing ones by extending this operation to apply to languages as follows.
- Let L_1 and L_2 be languages over some alphabet Σ . Then
$$L_1.L_2 = \{s.t \mid s \in L_1 \text{ and } t \in L_2\}$$

Defining new languages from old ones

- **n-times Concatenation:** If we apply concatenation to the same language by forming $L \cdot L$ there is no reason to stop after just one concatenation. For an arbitrary language L we define

$$L^n = \{s_1 \cdot s_2 \dots s_n \mid s_i \in L \text{ for all } 1 \leq i \leq n\}$$

We look at the special case

$$L^0 = \{s_1 \cdot s_2 \dots s_n \mid s_i \in L \text{ for all } 1 \leq i \leq n\} = \{\epsilon\}$$

since ϵ is what we get when concatenating 0 times.

Defining new languages from old ones

- **Kleene Star:** Sometimes we want to allow any (finite) number of concatenations of words from some language. The operation that does this is known as the Kleene star and written as $(-)^*$. The definition is:

$$\begin{aligned} L^* &= \{s_1 s_2 \dots s_n \mid n \in \mathbb{N}, s_1, s_2, \dots, s_n \in L\} \\ &= \bigcup_{n \in \mathbb{N}} L^n \end{aligned}$$

Note that

$$\emptyset^* = \bigcup_{n \in \mathbb{N}} \emptyset^n = \{\varepsilon\} = L^0$$

for all L , which may be a bit unexpected.

Language Definition

The languages can be defined in different ways:

- Descriptive Method
- Regular Expression
- Finite Automata
 - Deterministic Finite Automata
 - Non Deterministic Finite Automata

Descriptive Method

- The language is defined in the form of set, ***describing the conditions imposed on its words.***

Example:

- The language L of strings ***of odd length***, defined over $\Sigma=\{a\}$, can be written as
 - $L=\{a, aaa, aaaaa,.....\}$
- The language L of strings ***that does not start with a***, defined over $\Sigma =\{a,b,c\}$, can be written as
 - $L =\{\Lambda, b, c, ba, bb, bc, ca, cb, cc, baa,bab,bac,bba...\}$

Examples cont.

- The language L of ***strings of length 2***, defined over $\Sigma = \{0,1,2\}$, can be written as
 - $L = \{00, 01, 02, 10, 11, 12, 20, 21, 22\}$
- The language L of ***strings ending in 0***, defined over $\Sigma = \{0,1\}$, can be written as
 - $L = \{0, 00, 10, 000, 010, 100, 110, \dots\}$

Examples cont.

- The language **EQUAL**, *of strings with number of a's equal to number of b's*, defined over $\Sigma=\{a,b\}$, can be written as
 - $\text{EQUAL}=\{\Lambda, ab, ba, aabb, abab, baba, abba, \dots\}$
- The language **EVEN-EVEN**, of strings with even number of a's and even number of b's, defined over $\Sigma=\{a,b\}$, can be written as
 - $\text{EVEN-EVEN}=\{\Lambda, aa, bb, aaaa, aabb, abab, abba, baab, baba, bbaa, bbbb, \dots\}$
 - $\text{Even}=\{2,4,6,\dots\}$

Examples cont.

- The language **INTEGER**, of strings defined over $\Sigma=\{-,0,1,2,3,4,5,6,7,8,9\}$, can be written as
 - $\text{INTEGER} = \{\dots,-2,-1,0,1,2,\dots\}$
- The language **EVEN**, of strings defined over $\Sigma=\{-,0,1,2,3,4,5,6,7,8,9\}$, can be written as
 - $\text{EVEN} = \{\dots,-4,-2,0,2,4,\dots\}$

Examples cont.

- The language $\{a^n b^n\}$, of strings defined over $\Sigma=\{a,b\}$, as $\{a^n b^n : n=1,2,3,\dots\}$, can be written as
 - $a^n b^n = \{ab, aabb, aaabbb, aaaabbbb, \dots\}$
- The language $\{a^n b^n a^n\}$, of strings defined over $\Sigma=\{a,b\}$, as $\{a^n b^n a^n : n=1,2,3,\dots\}$, can be written as
 - $a^n b^n a^n = \{aba, aabbaa, aaabbbbaaa, aaaabbbbbaaaaa, \dots\}$
- The language $\{a^n b^m a^n\}$, of strings defined over $\Sigma=\{a,b\}$, as $\{a^n b^m a^n : n=1,2,3,\dots ; m=0,1,2,3,\dots\}$, can be written as
 - $a^n b^m a^n = \{aa, aaaa, aaaaaa, \dots, aba, aabaa, \dots\}$

Examples cont.

- The language **factorial**, of strings defined over $\Sigma=\{0,1,2,3,4,5,6,7,8,9\}$
 - **factorial**={1,2,6,24,120,...}
- The language **FACTORIAL**, of strings defined over $\Sigma=\{a\}$, as $\{a^{n!} : n=1,2,3,\dots\}$, can be written as
 - {a,aa,aaaaaa,...}.

It is to be noted that the language FACTORIAL can be defined over any single letter alphabet.

Examples cont.

- The language **DOUBLEFACTORIAL**, of strings defined over $\Sigma=\{a, b\}$, as $\{a^{n!}b^{n!} : n=1,2,3,\dots\}$, can be written as
 - **DOUBLEFACTORIAL**={ab, aabb, aaaaaabbbbbbb,...
- The language **SQUARE**, of strings defined over $\Sigma=\{a\}$, as $\{a^{n^2} : n=1,2,3,\dots\}$, can be written as
 - **SQUARE**={a, aaaa, aaaaaaaaaa,...

Examples cont.

- The language **PRIME**, of strings defined over $\Sigma=\{a\}$, as $a^p : p \text{ is prime}$, can be written as
- **PRIME**={aa,aaa,aaaaa,aaaaaaaa,aaaaaaaaaaaaa...}

An Important language

- PALINDROME

The language consisting of Λ and the strings s defined over Σ such that $\text{Rev}(s)=s$. It is to be denoted that the words of PALINDROME are called palindromes.

- Example:

For $\Sigma=\{a,b\}$, $\text{PALINDROME}=\{\Lambda, a, b, aa, bb, aaa, aba, bab, bbb, \dots\}$

- Language called vowels , contain all the vowels
- $L=\{a,e,i,o,u\}$
- Language containing loop keywords of c++
- $L=\{\text{for, while, do while}\}$
- Language defined over $\Sigma=\{a,b,c\}$ where length of strings is less than or equals to 2.
- $L=\{\epsilon, a,b,c, aa,ab,ac, ba,bb,bc, ca,cb,cc\}$
- Language defined over $\Sigma=\{a,b,c\}$ where length of strings is greater than or equals to 2.
- $L=\{aa,ab,ac, ba,bb,bc, ca,cb,cc, aaa,aab,aac,\dots\}$